INTERVIEW PREPARATION GUIDE

FinLedger — MongoDB Schema Design

A complete reference for explaining the database models, relationships, and design decisions of the FinLedger backend system in a technical interview.

Node.js | Express | MongoDB | Mongoose | JWT Auth | Double-Entry Ledger

📦 THE 5 MODELS — WHAT TO SAY

1. User Model

Collection: users

`email` `name` `password` `systemUser` `timestamps`

> The User model stores login credentials. Password is hashed automatically using a **pre-save hook** with bcrypt — so raw passwords are never stored. I also have **select: false** on the password field so it's never accidentally returned in queries. The **systemUser flag is immutable** to prevent privilege escalation attacks.

2. Account Model

Collection: accounts

`user → ref:User` `name` `type: SAVINGS | CHECKING | BUSINESS`
`status: ACTIVE | FROZEN | CLOSED` `currency: INR | USD | EUR` `getBalance() method`

> Each account belongs to a user via a reference. The most important part is the **getBalance() method** — it doesn't store balance anywhere. Instead it runs a **MongoDB aggregation** on the Ledger collection, summing all CREDITs and DEBITs for that account and subtracting to get the live, always-accurate balance.

3. Transaction Model

Collection: transactions

`fromAccount → ref:Account` `toAccount → ref:Account` `amount`
`status: PENDING | COMPLETED | FAILED | REVERSED` `idempotencyKey (unique)`

> A transaction records a fund transfer between two accounts. The **idempotencyKey is unique** — if the same request is retried (due to network failure etc.), MongoDB throws a duplicate key error, so money is **never debited twice**. Status flows from PENDING → COMPLETED or FAILED.

4. Ledger Model

Collection: ledgers — Core of the system

`account → ref:Account (immutable)` `transaction → ref:Transaction (immutable)`
`type: CREDIT | DEBIT (immutable)` `amount (immutable)`

> This is the **heart of the system**. Every transfer creates **two ledger entries** — a DEBIT on the sender's account and a CREDIT on the receiver's. All four fields are marked **immutable**, so they cannot be changed after creation. I also added pre-hooks on update, delete, and findOneAndUpdate that **throw an error** if anyone tries to modify a ledger entry — full tamper protection.

5. Token Blacklist Model

Collection: tokenblacklists

`token (unique)` `TTL Index: 3 days` `timestamps`

> When a user logs out, their JWT token is saved here. Every protected route checks this blacklist before processing the request. I added a MongoDB **TTL (Time-To-Live) index** — entries auto-delete after **3 days**, which matches the JWT expiry, so the collection never grows out of control.

Model Relationships — Say This at the End

```
User ——< Account ——< Ledger >—— Transaction
```

One user can have many accounts. Each account has many ledger entries. Each ledger entry points to one transaction. A single transaction **always has exactly two ledger entries** — that's the double-entry principle.

💡 **Interviewer follow-up:** *"Why not store the balance directly on the account?"*
Answer: *"Because storing balance creates a race condition risk if two transfers happen simultaneously. Computing it from immutable ledger entries is always accurate and gives a full, tamper-proof audit trail."*